



MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

Building a Smart Chatbot for tourism in VietNam using Retrieval Augmented Generation and Large Language Models

AIP491 Group 9	
Group Members	Pham Duy Anh - HE172206 Nguyen Thai Son - HE170136 Nguyen Thieu Dinh - HS173453 Luu Ba Quy Duong - HE172166
Supervisor	Tran Van Ha
Ext Supervisor	
Capstone Project code	AIP491

Abstract

Amidst the robust digital transformation of Vietnam’s tourism sector, where existing information retrieval solutions remain constrained by limited accuracy and a lack of real-time currency, this thesis presents the development and deployment of an intelligent Vietnamese tourism Chatbot leveraging Large Language Models (LLMs) integrated with a Retrieval-Augmented Generation (RAG) framework. The research commences with the construction of a standardized Knowledge Base comprising over 5,000 articles from authoritative sources such as Vietnamtourism and VNExpress. Concurrently, we address the challenge of training data scarcity by implementing a synthetic data generation pipeline utilizing GPT-4o-mini to produce tens of thousands of high-quality query-context pairs. Leveraging this dataset, we conducted comprehensive fine-tuning experiments and performance benchmarking across three representative embedding models: intfloat/multilingual-e5-base, sentence-transformers/all-mpnet-base-v2, and bkai-foundation-models/vietnamese-bi-encoder. The training process employed a Contrastive Learning approach optimized via the MultipleNegativesRankingLoss function. Empirical results demonstrate the superior domain adaptation capabilities of the fine-tuned BKAI-Vietnamese-Bi-Encoder, achieving a retrieval Accuracy@1 of 89.3% and ranking metrics (NDCG@10) exceeding 93%. This model serves as the backbone for the system’s Retriever module, deployed on ChromaDB infrastructure. By integrating Context Smoothing techniques with LLMs, the system effectively mitigates hallucination phenomena, delivering natural, precise, and verifiable responses to users.

Keywords: Tourism Chatbot, Retrieval-Augmented Generation (RAG), Large Language Models (LLM), Embedding Fine-tuning, BKAI-Vietnamese-Bi-Encoder, Vector Database.

Acknowledgments

First and foremost, we would like to express our deepest gratitude to our supervisor, Mr. Tran Van Ha. His dedicated guidance, sharp orientation, and profound professional knowledge served as the guiding compass that helped the group navigate through the most challenging phases of this project. Beyond his role as an instructor, his patience and timely encouragement provided immense motivation for us to refine and complete this product to the best of our abilities.

We also extend our sincere thanks to FPT University for fostering a dynamic and challenging academic environment. The support from the faculty members within the department served as a valuable foundation of knowledge that accompanied us throughout the implementation process. It was the rigorous training standards, combined with the rich infrastructure and resources, that created favorable conditions for us to research and realize our ideas. Furthermore, the learning environment here has forged our resilience and adaptability under pressure—prerequisites for the successful completion of this project.

Additionally, we deeply cherish the companionship of our friends. The discussions, knowledge sharing, and candid constructive criticism helped the group view problems from multiple dimensions, thereby significantly improving the quality of this thesis.

Finally, our most profound gratitude goes to our families. We thank our parents for always being our solid support system, for their understanding, and for creating the best possible conditions for us to devote ourselves fully to our studies and research.

Contents

1	Introduction	6
1.1	Motivation	6
1.2	Objectives	6
2	Related Works	7
2.1	Current AI-Integrated Tourism Applications and Websites	7
2.2	Published Studies on Tourism Chatbots and LLM/RAG Applications in Other Domains	7
2.3	Limitations of Existing Solutions and Proposed Improvements	7
3	Theoretical basis	8
3.1	Overview	8
3.2	Data preparation	8
3.3	Fine - Tuning Embedding Models	8
3.4	Retrieval	9
3.5	Generation	9
4	Dataset	9
4.1	Data Sources and Collection Method	9
4.2	Data Preprocessing	10
4.3	Segmentation Method	10
4.4	Data Preparation for Embedding Model Fine-tuning	11
4.5	Data Noise Filtering using Cross-Encoder	11
5	Select the embedding model based on the fine-tuning results	12
5.1	Experimental Models	12
5.2	Fine tune results on models	19
6	Construction of Vector Database and Storage System	25
6.1	Select vector database	25
6.2	Data Structure and Organization	25
6.3	Interaction History Storage (MongoDB):	25
7	System Implementation	26
7.1	Query Processing	26
7.2	Retrieval and Context Reconstruction Module	27
7.3	Response Generation	27
7.4	System Demonstration	28
8	Conclusion	30

List of Figures

Figure 1	System architecture overview	8
Figure 2	Data Structure After Chunking	11
Figure 3	Comparison chart of evaluation score distribution (Dataset 1).	12
Figure 4	Comparison chart of evaluation score distribution (Dataset 2).	12
Figure 5	Architecture diagram and data processing flow of the Intfloat/multilingual-e5-base model . .	13
Figure 6	Architecture and Data Processing Flow of the BKAI-Vietnamese-Bi-Encoder	14
Figure 7	Architecture and Data Processing Flow of the All-mpnet-base-v2 model.	15
Figure 8	Chart comparing actual performance before and after fine-tuning the intfloat/multilingual-e5-base model	19
Figure 9	Chart comparing actual performance before and after fine-tuning the model bkai-foundation-models/vietnamese-bi-encoder	20
Figure 10	The chart comparing the actual performance of the all-mpnet-base-v2 model before and after fine-tuning	21
Figure 11	Chart comparing actual performance before and after fine-tuning the intfloat/multilingual-e5-base model	22
Figure 12	Chart comparing actual performance before and after fine-tuning the model bkai-foundation-models/vietnamese-bi-encoder	23
Figure 13	The chart comparing the actual performance of the all-mpnet-base-v2 model before and after fine-tuning	24
Figure 14	Logical structure of a record in ChromaDB	25
Figure 15	Cấu trúc Prompt phân loại ý định người dùng (Small Talk vs. Tourism Intent)	26
Figure 16	System Prompt for Contextual Query Rewriting	27
Figure 17	Configuration of the System Prompt for the Travel Assistant Persona	28
Figure 18	Chatbot User Interface.	28

1 Introduction

1.1 Motivation

In recent years, the Vietnamese tourism industry has witnessed a robust recovery and significant growth, establishing itself as a key economic spearhead and attracting substantial attention from both domestic and international travelers. According to the Vietnam National Authority of Tourism, in 2023, Vietnam welcomed 12.6 million international visitors, nearly 3.5 times higher than in 2022, surpassing the initial target by 57%, while domestic tourism reached 108 million trips, generating a total revenue of 678 trillion VND [1]. This upward trajectory continued impressively into 2024; in the first ten months alone, international arrivals exceeded 14.1 million, marking a 41.3% increase year-on-year and approaching the pre-pandemic peak of 2019 [2]. The rapid advancement of technology has catalyzed a shift from traditional tourism models to self-sufficient and smart tourism. Travelers increasingly rely on digital platforms for itinerary planning, making the demand for accessible information regarding destinations, culture, cuisine, transportation, and accommodation more critical than ever.

However, the explosion of internet information has introduced a dual challenge: data overload and fragmentation. Details concerning locations, culture, and pricing are often scattered across various websites, forums, and social networks. This data frequently lacks verification and is subject to real-time fluctuations, creating a significant barrier for tourists seeking accurate and reliable information efficiently.

Existing tourist support solutions fall primarily into two categories, both of which exhibit distinct limitations. First, traditional search engines typically return a list of hyperlinks, compelling users to manually navigate, filter, and synthesize information. Meanwhile, traditional rule-based chatbots are often overly rigid, restricted to predefined scripts, and lack the capability to comprehend complex contexts or facilitate natural interactions.

Second, the emergence of Large Language Models (LLMs) such as GPT and Gemini has introduced remarkable conversational fluency. However, when applied to a specialized domain like tourism, these models frequently encounter severe issues with "hallucination"—a tendency to generate plausible-sounding but factually incorrect information. They often fabricate details when relevant data is absent from their training corpus, particularly regarding specific, detailed, or real-time updates (e.g., ticket prices, opening hours, local events) within the context of Vietnamese tourism.

In response to this context, the project "Vietnamese Tourism Information Q&A Chatbot" is proposed to directly address these challenges. The primary rationale for this project is to apply the cutting-edge Retrieval-Augmented Generation (RAG) technology to a field with high practical demand. Instead of relying on the LLM to "invent" responses, the RAG technique enables the system to Retrieve accurate information from a reliable Knowledge Base of Vietnamese tourism. This information is then used to Augment the context, allowing the LLM to Generate the final response.

This approach not only resolves the critical "hallucination" weakness of LLMs but also provides tourists with a 24/7, instantaneous, accurate, and personalized information channel. The project holds significant value in enhancing the tourist experience, optimizing information delivery, and contributing to the digital transformation of Vietnam's tourism industry.

1.2 Objectives

In this study, we focus on developing a robust Vietnamese tourism Q&A Chatbot designed to address the limitations of Large Language Models regarding local knowledge and accuracy, employing an enhanced Retrieval-Augmented Generation (RAG) architecture. The research centers on constructing an intelligent tourism chatbot grounded in a standardized knowledge base, which is aggregated from authoritative sources and meticulously processed using specific Vietnamese Natural Language Processing techniques such as word segmentation and noise removal. To mitigate training data constraints, we implemented an automated synthetic data generation pipeline utilizing GPT-4o-mini, establishing the foundation for fine-tuning a specialized embedding model via Contrastive Learning and the MultipleNegativesRankingLoss function; this approach optimizes the model's deep semantic understanding of domain-specific queries. Architecturally, the system employs an Adaptive RAG framework featuring a user intent classification mechanism, enabling flexible transitions between casual conversation and intensive knowledge retrieval to optimize computational resources. Finally, the pipeline is completed by a high-performance Vector storage and retrieval system utilizing Cosine Similarity, integrated with Context Smoothing techniques and the capabilities of the GPT-4o-mini Large Language Model to ensure that the generated responses are consistently accurate, natural, and informative.

2 Related Works

2.1 Current AI-Integrated Tourism Applications and Websites

Currently, numerous applications and websites in the tourism market have integrated AI chatbots; however, when applied to the Vietnamese context, each solution exhibits distinct advantages and limitations.

Specifically, Layla (Roam Around) stands out as a comprehensive travel assistant, utilizing LLM technology and a unique "Visual Discovery" feature to suggest itineraries and book services[3]. However, it struggles to comprehend local slang or introduce "niche" destinations deeply rooted in Vietnamese cultural identity. Similarly, Expedia and Booking.com have directly integrated ChatGPT to optimize hotel and flight booking processes[4]. Yet, due to their heavily commercial nature, suggestions often favor major partners and lack depth regarding backpacking experiences or street food culture.

In the domestic market, Chatbot Danang Fantasticity is a pioneering solution supporting tourists via Facebook Messenger. However, its reliance on legacy NLP generations and decision-tree scripts renders it rigid and inflexible in conversation. Finally, Traveloka (Teresa) focuses on customer service and handling after-sales tasks with high accuracy, but its operational scope is limited to commercial transactions, lacking the capability to provide experiential advice or personalized itineraries. Existing solutions are either too generalized, lacking local depth, or technologically rigid, creating a significant gap for the development of a specialized tourism chatbot that deeply understands Vietnamese culture and language.

2.2 Published Studies on Tourism Chatbots and LLM/RAG Applications in Other Domains

In the realm of academic research, the application of chatbots in tourism has emerged as a topic of significant interest aimed at enhancing customer experience. Calvaresi et al. (2021) [5] conducted a comprehensive systematic review on chatbots in tourism and hospitality, indicating that although the number of applications is increasing, most current systems still operate based on rule-based scripts and lack the ability to handle complex conversations or deep context understanding. Delving into the behavioral aspect, the study by Pillai and Sivathanu (2020) [6] on tourist acceptance of AI chatbots demonstrated that "perceived trust" and "perceived usefulness" are key factors, posing an urgent requirement for developers to ensure the absolute accuracy of the provided information. Technically, to overcome keyword matching limitations, Sperlí (2018) [7] proposed using ontologies and the Semantic Web to build smarter chatbots; however, the automated data scalability of this method remains limited. Most recently, amidst the explosion of Generative AI, Buhalis et al. (2023) [8] stated that Large Language Models (LLMs) like ChatGPT will completely reshape the tourism industry through hyper-personalization capabilities, but simultaneously issued a stern warning regarding the risk of information hallucination.

Beyond the tourism sector, the integration of Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) has catalyzed a revolution across various essential industries, proving the efficacy of domain-specific model adaptation. In the medical field, Li et al. (2023) introduced ChatDoctor [9], a model fine-tuned on the LLaMA architecture using a large-scale medical knowledge graph. By retrieving real-time medical databases, this model significantly enhances diagnostic accuracy and patient consultation compared to general-purpose models. In the financial sector, Wu et al. (2023) presented BloombergGPT [10], a pioneer model trained on a massive proprietary financial dataset, effectively addressing complex market analysis and specialized text processing tasks, where accuracy is considered a paramount factor.

Furthermore, in education, Kasneci et al. (2023) highlighted the potential of LLMs in personalizing learning paths and acting as intelligent virtual tutors; however, the authors also emphasized the necessity of RAG to ensure academic integrity and factual correctness [11]. Similarly, in agriculture, recent studies such as Rezayi et al. (2023) have explored using domain-specific Transformers to provide precise farming techniques and pest control advice to farmers, helping overcome language barriers and expert shortages [12]. These interdisciplinary advancements collectively underscore a common conclusion: while general LLMs are powerful, integrating them with specialized knowledge retrieval mechanisms (RAG) is a prerequisite for deploying reliable and high-stakes applications.

2.3 Limitations of Existing Solutions and Proposed Improvements

Compared to existing products, our project differentiates itself significantly through its Hyper-Local Language Understanding and absolute reliability[13]. While international applications often struggle with slang such as "săn mây" (cloud hunting), "check-in", or regional culinary nuances, our chatbot can subtly distinguish between local characteristics. It provides accurate, source-cited information thanks to the RAG system[14], effectively eliminating the "hallucination" phenomenon common in older LLM models[15].

Looking towards the future, the system is positioned to become a comprehensive and sophisticated travel assistant. It aims to feature capabilities for real-time risk alerts (such as flooding or price gouging), suggesting "Hidden Gems" from reputable blogger communities rather than commercialized locations. Furthermore, the system is designed to flexibly adjust schedules based on weather conditions (Dynamic Itinerary) and provide profound cultural guidelines (such as dress codes at spiritual sites), delivering the safest, most authentic, and complete travel experience for tourists.

3 Theoretical basis

3.1 Overview

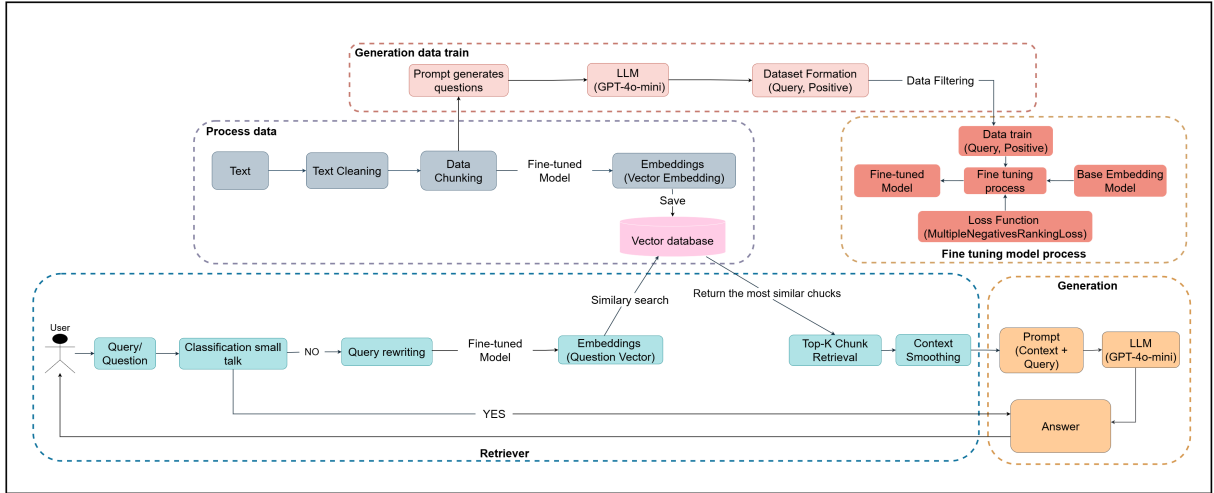


Figure 1: System architecture overview

3.2 Data preparation

The data preparation process is designed to create a high-quality dataset for fine-tuning the embedding model of the RAG system. Initially, raw textual data are collected from multiple reliable sources related to Vietnamese tourism, including descriptions of destinations, historical background, cultural information, cuisine, and transportation guidelines. These texts are then cleaned and preprocessed before being passed to the next stage. Subsequently, the entire corpus is segmented into smaller text chunks following fixed rules, typically ranging from 100 to 300 words. An overlap of 50–80 words between adjacent chunks is applied to ensure semantic continuity across segments. These chunks serve as positive documents, forming the basis for question generation and acting as ground-truth labels during training. To construct the training data, each chunk is fed into a standardized prompt to prompt the GPT-4o-mini model to generate a set of corresponding questions that reflect realistic user queries. The generated questions are manually reviewed to remove those that are irrelevant, unnatural, or duplicated. After this filtering step, each high-quality question is paired with its source chunk to form query–positive data pairs. Once the query–positive pairs are formed, the dataset undergoes an additional quality evaluation stage to ensure semantic consistency. Data pairs with low relevance, questions that do not accurately reflect the content of the corresponding text, or duplicated pairs are removed. This filtering process enhances the reliability of the training dataset and provides a robust foundation for fine-tuning the embedding model.

3.3 Fine - Tuning Embedding Models

We implement a fine-tuning process built upon a foundational Base Model. The input for this phase is a training dataset comprising "Query-Positive" pairs, automatically generated from the original data corpus utilizing the GPT-4o-mini model. The core of the training phase involves updating the model weights via a Contrastive Learning mechanism employing the MultipleNegativesRankingLoss function. Specifically, within each data batch, the model learns to optimize vector distances by minimizing the distance between the query vector and its corresponding positive document vector (Positive sample), while simultaneously maximizing the distance from all other documents in the batch (treated as In-batch Negatives). The resulting output is an optimized embedding

model that plays a pivotal role in two critical stages: encoding data for ingestion into the Vector Database and processing real-time user queries. Thanks to this mechanism, the system overcomes the limitations of surface-level keyword matching methods, enabling the chatbot to accurately capture semantic relationships. This ensures precise information retrieval even when users employ slang or ambiguous expressions.

3.4 Retrieval

First, the system receives a query submitted by the user. It does not immediately initiate the search process. Instead, it classifies whether the query is conversational (chitchat) or not. YES Branch: If the query is conversational, the system proceeds directly to GPT-4o-mini to generate a natural response, bypassing the data retrieval phase. NO Branch: If the query requires domain-specific knowledge, the system proceeds to the RAG pipeline described below.

The user's text query is encoded into a numerical vector using the specifically fine-tuned embedding model. This ensures that the semantic representation of the user query aligns with how the data has been stored in the database. The system compares the query vector against millions of vectors stored in the Vector DB. It utilizes the Cosine Similarity algorithm to calculate the angular distance between vectors. Vectors with the smallest angles indicate the text passages most semantically similar to the query. Instead of retrieving all matches, the system selects only the Top-K results (specifically the Top-10) with the highest similarity scores.

3.5 Generation

The system aggregates the retrieved information pieces into a complete "prompt" to send to GPT-4o-mini. The structure typically follows this format: "You are a virtual travel assistant. Below is the context information found: [Content from Smooth context in Step 2]. Based on this, please answer the user's question: [Original Query]. If the information is unavailable, state that you do not know." If Step 2 identified the query as conversational, the prompt is simpler: "Please respond to this user greeting in a friendly manner." The GPT-4o-mini model receives the formatted prompt. It leverages its linguistic capabilities to comprehend the provided context and synthesize a coherent response with correct Vietnamese grammar. The final response is then returned to the user.

4 Dataset

4.1 Data Sources and Collection Method

The dataset in this study was developed based on collecting content from four large and highly reliable tourism information sources in Vietnam, including VNExpress[16], Vietnamtourism[17], iVIVU[18], and Dulichviet[19]. The selection of these sources aims to ensure the data has high credibility, diverse content, and comprehensively covers localities nationwide. From VNExpress, 111 articles were collected; from Vietnamtourism – the portal of the Vietnam National Administration of Tourism – 975 articles were obtained. Data from iVIVU and Dulichviet were collected in parallel with comparable scale, aiming to ensure full representation for all provinces and cities. In total, the data collection process yielded over 5000 original articles from the four platforms.

The data crawling method was executed via custom Python scripts, specifically designed for each website to be compatible with the specific HTML structure of each source. The crawling process included retrieving article lists within tourism categories, fetching individual article content at the detail page level, and extracting core components such as titles, main content, and source URLs. Simultaneously, the crawler removed unnecessary UI elements such as advertisements, navigation menus, suggestion blocks, or auto-inserted content to ensure that only text serving knowledge analysis was collected. Each data source was processed independently, ensuring traceability and cross-referencing capabilities when merged into the aggregate dataset.

Statistically, the collected dataset consists of over 5000 articles, evenly distributed across the four platforms. The data scale reaches hundreds of thousands of sentences and paragraphs after sentence segmentation and content organization, creating a large-capacity knowledge base with good coverage for each tourist location. Furthermore, each data entry is tagged with origin information, including the article title and original URL, allowing for reverse verification of content when necessary and ensuring the transparency of the entire dataset.

The data structure is organized systematically by geographical unit. Right from the collection phase, articles were labeled according to the province or city the content mentions, and grouped by the three regions: North, Central, and South. This organization helps maintain consistency among knowledge sources related to each region, while facilitating information retrieval, model evaluation, and integration into the RAG system. Grouping by geographical unit also creates a crucial foundation for subsequent processing stages such as building a Vector Database or deploying embedding models.

4.2 Data Preprocessing

After collecting articles from four data sources, we store each article as a separate record in the database. Each record is categorized into specific sections for easier management and retrieval, such as “Chơi ở đâu” “Ăn gì ở An Giang” “Đặc sản ở An Giang” “Điểm đến” ... This categorization helps the system easily organize information by topic and enhances efficiency when applied to RAG systems or embedding models.

Next, all data undergoes a preprocessing pipeline. The processing steps include removing redundant information, special characters, HTML tags, advertisements, paragraphs unrelated to tourism, as well as very short articles that lack valuable information. At the same time, duplicate data is detected and removed to ensure a diverse and high-quality knowledge base.

After automatic filtering, the remaining data is manually reviewed to eliminate disjointed paragraphs, content lacking context, or inaccurate information. All data is also standardized in format, with unnecessary whitespace removed and structure unified, making it ready for subsequent processing steps.

This workflow helps improve the quality of the corpus, ensuring that the RAG system does not retrieve noisy or low-value information, while also enabling embedding models to generate accurate and effective semantic representations.

Table 1: Data Analysis Before and After Cleaning

Category	Before Cleaning	After Cleaning	Notes / Processing
Number of Records	5,316	5,104	Removed duplicate and invalid records
Duplicate Records	45	0	Kept only one record per content
Incomplete Records	47	0	Removed records with empty or incomplete content
Short Texts	120	0	Removed texts shorter than the minimum threshold
Special Characters / HTML	Many	None	Cleaned using regular expressions
Average Length (words)	52	89	Cleaned texts are more complete and context-rich

4.3 Segmentation Method

To ensure the model effectively processes long texts and optimizes information retrieval, we apply a text segmentation technique based on sentence structure and token length. Splitting text by sentence boundaries not only helps preserve semantics but also ensures each data segment has an appropriate length, avoiding mid-sentence truncation and the loss of critical information. The goal of the chunking method is to balance between retaining full context and limiting segment size to avoid overloading the model. The First Chunking Method: For our dataset, the selected method is sentence-boundary-based chunking combined with a dynamic length balancing mechanism, where each chunk is maintained at approximately 150 words and no fewer than 50 words. The dataset comprises texts of varying lengths and structures; some are long with complex sentences, while others are short and simple. Consequently, segmenting by sentence boundaries preserves the meaning of individual sentences, avoiding mid-sentence truncation that leads to loss of critical information. After splitting sentences and grouping them into chunks approaching the 150-word[20] threshold, the system reviews and handles overly short chunks by adjusting the length through “borrowing sentences” from the preceding chunk or merging chunks if necessary.

This approach balances long and short segments within the data, ensuring every chunk contains sufficient context for the embedding model to fully comprehend the content, while eliminating overly short segments that lack adequate information. Preserving sentence boundaries and unifying chunk lengths enhances the quality of embedding vectors and ensures input consistency for models such as E5, BKAI, or MPNet. As a result, the RAG system can retrieve information more accurately, ensuring data segments have high utility value in both training and inference phases, while maximizing the potential of our specific dataset characteristics.

The Second Chunking Method: In the initial stage, we applied a segmentation method based on sentence structure and balanced length[21, 22]. This approach helps maintain basic coherence and ensures that each segment does not exceed the allowed token limit. However, this method revealed several important limitations. Since there is no overlap between segments, when critical content lies at the boundary between two segments, the model may lose semantic connections. This reduces the effectiveness of semantic reasoning, especially when processing texts with tightly linked sentences. Additionally, the length of segments can vary significantly due to the mechanism of borrowing or merging sentences. This inconsistency results in unstable embedding representations. The method also struggles with very long sentences, which may force the system to create unusually large segments, leading to data imbalance and a decrease in representation quality.

To address these limitations, we experimented with an alternative segmentation method based on a sliding window with token overlap[23, 24]. The rationale for this approach stems from the need to preserve continuous

context between segments. When data is used for semantic search and RAG systems, dividing texts into independent segments can easily result in the loss of important information or disrupt the content flow. In the second method, each segment is created with a maximum length of approximately 300 tokens and slides over the text in fixed steps of 220 tokens[25]. Consequently, consecutive segments always share an overlapping region of nearly 80 tokens, ensuring that critical sentences at segment boundaries are repeated and fully captured by the embedding model. This mechanism allows the model to better capture semantic relationships that span multiple sentences while significantly reducing context loss commonly observed in the first method.

Creating fixed-size sliding windows also contributes to a more consistent input dataset[25]. Statistics across the dataset after applying this method show that most segments fall within the range of 230 to 290 tokens, with only a very small proportion of segments discarded due to being shorter than 50 tokens. This stability in segment length reduces variation in data distribution and supports models such as E5, BKai, and MPNet in generating more accurate semantic representations.

```
{
  "title": "output_paragraph_1",
  "line_first_sentence": "An Giang mùa nào đẹp."
  "passage": "An Giang mùa nào đẹp. Ở An Giang: Từ tháng 3 đến tháng 5....",
  "id": 9,
  "len":
}
```

Figure 2: Data Structure After Chunking

4.4 Data Preparation for Embedding Model Fine-tuning

To construct the training dataset for the embedding model, we employed a Data Augmentation technique by utilizing the OpenAI GPT-4o-mini API to automatically synthesize queries corresponding to each prepared text segment.

In the first chunking method, a single query is generated for each text segment. The process ingests source text and employs a specialized prompt to instruct the model to produce a unique Vietnamese query, derived strictly from the original content. Data checkpoints are saved periodically, enabling the system to automatically resume operations in case of interruptions. Consequently, this phase yielded a final dataset comprising 25,153 query-positive pairs.

In the second chunking approach, each passage is expanded to a larger size 300 words, so we use the GPT-4o-mini model to generate five Vietnamese query questions for each passage. Unlike the first method—where a single query is sufficient for 150-word chunks with narrow semantic scope—the longer 300-word passages contain more ideas and layers of information, making multiple queries necessary to fully cover their content. All questions are generated strictly from the original passage to ensure accuracy and semantic consistency. The resulting data pairs are sequentially stored in a PKL file, with a total of 75,355 pairs. Generating five queries per passage increases semantic diversity, enriches the training data without expanding the original corpus, improves fine-tuning performance for ranking and retrieval tasks by providing multiple relevant signals for the same positive passage, and supports more effective negative sampling. As a result, the second dataset becomes more informative and better suited for models requiring strong and diverse retrieval capabilities.

4.5 Data Noise Filtering using Cross-Encoder

Training data generated automatically (by GPT-4) may contain noise. The goal of this step is to use independent evaluation models to eliminate low-quality data pairs, ensuring the RAG model does not learn incorrect knowledge later. The data filtering process involves evaluation using a Cross-Encoder (specifically the BAAI/bge-reranker-v2-m3 model). The system inputs the *Query* and *Passage* pair directly into the model for scoring, then proceeds to filter based on a threshold. Unlike Bi-Encoders which encode separately, the Cross-Encoder method concatenates the question and passage into a single sequence in the format *[CLS] Query [SEP] Passage [SEP]*. This sequence is then passed through Transformer layers to process deep semantic relationships. The output scoring mechanism is represented by the formula:

$$\text{Score} = \sigma(W \cdot \text{CLS}_{\text{output}} + b) \quad (1)$$

Where:

- CLS_{output} : The vector representing the entire sentence pair at the final layer.
- W, b : The weights and bias of the Linear layer projecting to 1 dimension (scalar).
- σ : The activation function (usually Sigmoid) to output the similarity score.

Wherein the final classification layer calculates the probability of whether the text passage truly answers the question. The returned score is a real number, representing the confidence level of the data pair.

Analysis of Data Filtering Results: Evaluation results on the two datasets show that the semantic generation quality of GPT-4 is quite high; however, there is still a difference in the retention rate between different datasets.

Dataset 1: With an initial sample dataset of 25,153 pairs, the average Reranker score reached a very high level (96,4%). The retention rate reached 93,94%, with only about 1.525 samples being rejected, including the removal of duplicate data. The remaining data after discarding was 23,628.

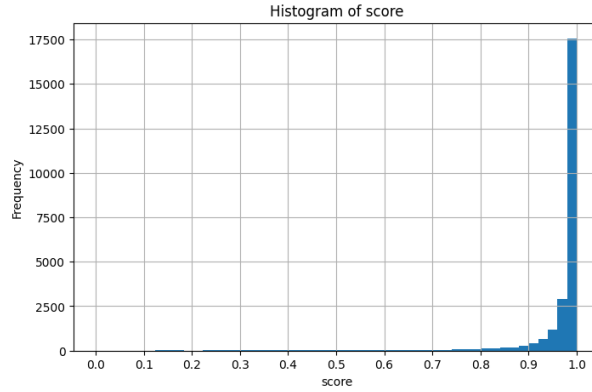


Figure 3: Comparison chart of evaluation score distribution (Dataset 1).

Dataset 2: With an initial sample dataset of 75,355 pairs, the average score decreased slightly to 0,8569. The retention rate was 86.97%, equivalent to removing about 9.816 noisy samples to ensure quality. The remaining data after discarding was 65.539.

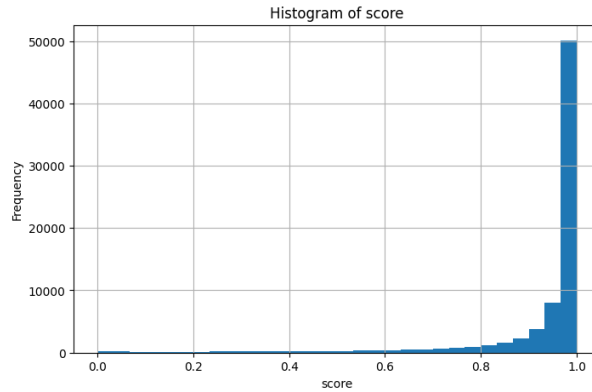


Figure 4: Comparison chart of evaluation score distribution (Dataset 2).

5 Select the embedding model based on the fine-tuning results

5.1 Experimental Models

Intfloat/multilingual-e5-base [25] is an advanced multilingual model capable of comprehending and processing text from over 200 languages. Rather than simply translating from one language to another, it understands the "deep underlying semantics" of the text, converting it into embedding vectors that can be utilized for search and information retrieval tasks. Fundamentally, E5-base is a Transformer Encoder model built upon the XLM-RoBERTa (base) architecture. Its backbone leverages XLM-RoBERTa, which is designed to grasp the semantics of multilingual text. The E5 encoder block is constructed with 12 processing layers; each layer features a hidden size of 768

dimensions and 12 attention heads, totaling approximately 270 million parameters. The core mechanism of E5 is Self-Attention, which enables the model to understand the relationships between words in a sentence. Additionally, it is Bidirectional, reading text in both directions simultaneously to capture the complete context.

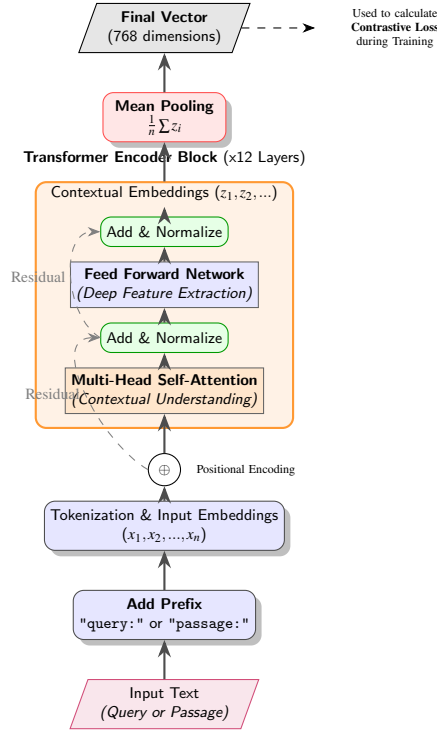


Figure 5: Architecture diagram and data processing flow of the Intfloat/multilingual-e5-base model

Input Stage raw data is retrieved from the training data file, followed by Prefix Addition, where the system automatically concatenates the strings. E5 Encoder Stage: Positional Encoding: Helps the model know that the word "Hà" stands before "Long". Without it, "Long Hà" and "Hà Long" would be identical. The Self-Attention Mechanism calculates the relationship between every word with one another to generate context. Add & Normalize + Feed Forward helps refine features and helps the training process avoid errors. The output at this stage is still a sequence of vectors. Each word has a separate vector. Post-processing Stage this is a crucial step to transform the Transformer's output into something the Vector DB can understand. The Problem A 10-word sentence results in 10 vectors (z). However, the Vector DB only accepts a single unique vector for a document. The Solution the computer will average all the vectors of the words. The result obtained is a single unique vector (768 dimensions) containing the entire meaning of that sentence. Training Stage after the Vector is generated, it enters the Loss Function. Here, the system compares the Question Vector with the Passage Vector. If they are not yet similar, the system reports an error (High Loss) and forces the blocks in Stage 2 to adjust weights so that it performs better next time. In Summary the E5 model is not merely a standalone Transformer Encoder block. To operate effectively for tourism data, the processing pipeline must include a Prefixing step at the input to orient the context (query/passage) and a Mean Pooling layer at the output to condense information into a single representative vector.

BKAI-Vietnamese-Bi-Encoder: In the Retrieval-Augmented Generation (RAG) system, the "Retriever" component acts as the backbone, determining the overall system accuracy; however, processing Vietnamese tourism data presents significant challenges due to semantic complexity and the ambiguity of named entities. To rigorously address this, we selected the BKAI-Vietnamese-Bi-Encoder, a Pre-trained Language Model developed by the International Research Institute for Artificial Intelligence and Data Science (BKAI) at Hanoi University of Science and Technology. Recognized as a State-of-the-art (SOTA) model for Vietnamese information retrieval, it outperforms general multilingual models by leveraging a massive native training corpus and an optimized architecture described in the BKAI Foundation Models paper [26]. Furthermore, its backbone is built upon PhoBERT, which utilizes a specialized tokenizer to effectively eliminate the compound word fragmentation issue common in international solutions [27]. This design ensures the complete preservation of semantic nuances in place names and

cultural terms, while its parallel Bi-Encoder mechanism provides the optimal speed required for real-time retrieval [28].

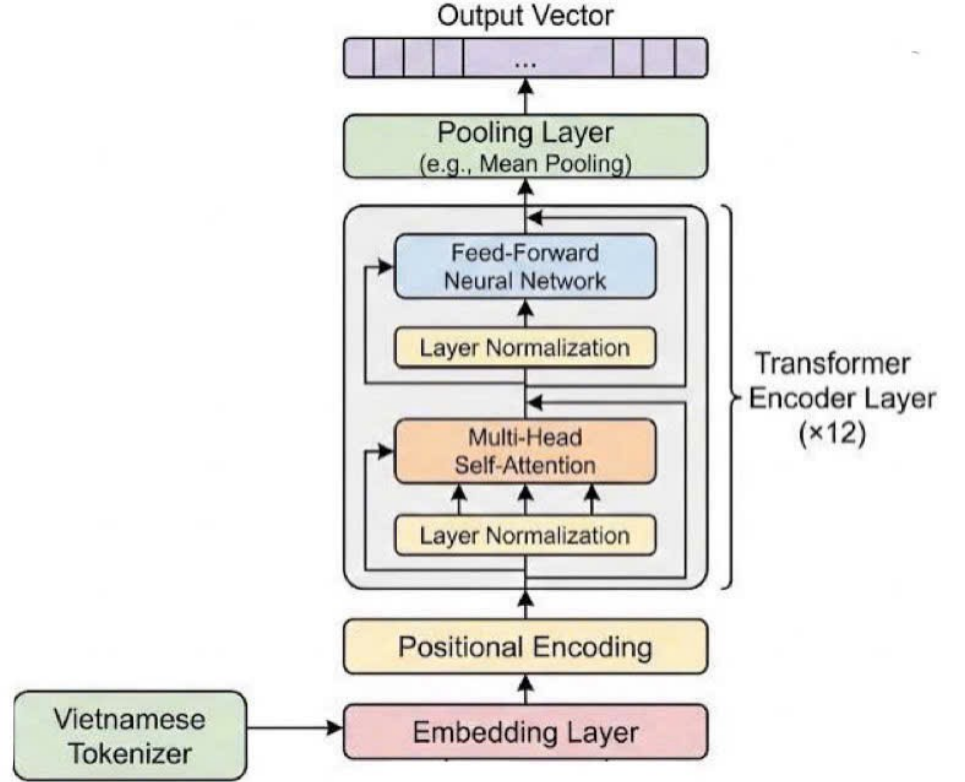


Figure 6: Architecture and Data Processing Flow of the BKAI-Vietnamese-Bi-Encoder

BKAI’s Transformer Architecture: The processing workflow of the BKAI-Vietnamese-Bi-Encoder commences with the transformation of input text via a specialized Vietnamese Tokenizer. This component identifies and preserves the structure of compound words and specific toponyms, avoiding the fragmentation issues typical of conventional models. Subsequently, these tokens are encoded into vectors and augmented with Positional Encoding, enabling the model to perceive word order and distinguish between varying contexts despite identical vocabulary usage. The data then propagates through a stack of 12 Transformer Encoder layers, where Self-Attention and Multi-Head Attention mechanisms facilitate a multidimensional analysis of semantic relationships between words. This process allows the model to deeply comprehend the intricate associations between tourism entities and their corresponding service attributes. Finally, after passing through Feed-Forward networks and normalization layers at each stratum, the output is aggregated (via Pooling techniques) into a single dense vector. This vector encapsulates the condensed semantic essence of the text, serving as the basis for precise similarity calculations and information retrieval.

MPNet (Masked and Permuted Pre-training Network)[22] is a language model introduced by Microsoft in 2020, built upon the Transformer Encoder architecture and designed to combine the strengths of two predecessor approaches: Masked Language Modeling from BERT[21] and Permutation Language Modeling from XLNet[29]. By employing the Permuted Masked Language Modeling (PMLM) technique, MPNet effectively avoids the information leakage issue present in XLNet while eliminating the training–inference discrepancy caused by the use of the [MASK] token in BERT. As a result, the model is able to capture long-range dependencies more effectively and learn more accurate semantic representations, leading to significant improvements in tasks such as semantic search, information retrieval, sentence embedding[23], and natural language understanding. Despite maintaining a model size comparable to base-scale architectures, MPNet consistently achieves superior performance across a wide range of standard benchmarks.

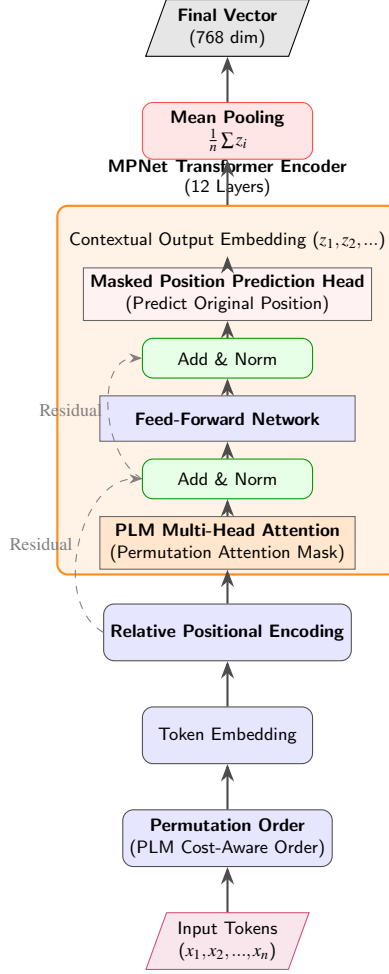


Figure 7: Architecture and Data Processing Flow of the All-mpnet-base-v2 model.

For the dataset used in this project, the inputs to MPNet consist of queries and positive documents. Each query or positive document is tokenised into a sequence of tokens $x_1, x_2, \dots, x_n$. Before being fed into MPNet, a permutation order (PLM cost-aware order) is applied to generate an optimal token permutation, allowing the model to learn token dependencies without information leakage. Each token is then mapped to a vector representation through token embeddings combined with Relative position encoding[30], which allows the model to capture relative distances between tokens—an aspect that is particularly important for encoding information related to time, location, and activities. The token sequence is passed through multi-head self-attention layers, where each token attends to preceding tokens according to the permutation order. This process is followed by Add & Norm operations and feed-forward networks and is repeated across 12 encoder layers to extract deep contextual features. In the final stage, a masked position prediction head is used to recover the original positions of the permuted tokens, enabling the model to jointly learn semantic representations and natural word order. The contextual embeddings resulting $z_1, z_2, \dots$ are aggregated using mean grouping to form a 768-dimensional vector. In this project, the query embedding is compared with the embeddings of positive documents to perform retrieval-based question answering, thereby identifying the most relevant information passages for generating accurate responses.

Evaluating the retrieval system requires simple yet effective metrics that accurately reflect the objective of chunk retrieval. To ensure rigorous assessment, we adopt standard Information Retrieval evaluation protocols as described in [31]. The following evaluation metrics are used:

Mean Reciprocal Rank (MRR): That measures the overall ranking quality by evaluating the position of the first correct result. The core idea of MRR is to “reward” systems that place the correct result at a high rank and “penalize” systems where the correct result falls to a lower rank. The MRR formula is expressed as follows:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (2)$$

Where:

- $|Q|$ is the total number of queries in the test set.
- $rank_i$ is the position (rank) of the first relevant result for the i -th query.
- $\frac{1}{rank_i}$ is the reciprocal rank score for the i -th query (if no correct result is found, this value is 0).

nDCG (Normalized Discounted Cumulative Gain): nDCG is a metric measuring the relevance of a ranked list of results. It accounts for the position of relevant items in the list, prioritizing higher relevance items placed at the top positions. nDCG is calculated by normalizing DCG (Discounted Cumulative Gain) by IDCG (Ideal Discounted Cumulative Gain).

$$NDCG_p = \frac{DCG_p}{IDCG_p} \quad (3)$$

DCG (Discounted Cumulative Gain) at position p :

$$DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log_2(i+1)} \quad (4)$$

Where rel_i is the relevance of the result at position i .

IDCG (Ideal Discounted Cumulative Gain) at position p : IDCG is the maximum achievable DCG, calculated by sorting items by their ideal relevance from high to low.

$$IDCG_p = \sum_{i=1}^p \frac{2^{rel_{(i)}} - 1}{\log_2(i+1)} \quad (5)$$

Where:

- $NDCG_p$: The nDCG value calculated up to position p . In your results, it is NDCG@10, meaning $p = 10$.
- rel_i : Relevance Score of the result at position i . Usually a discrete numerical value (e.g., 0 = not relevant, 1 = relevant, 2 = highly relevant).
- $2^{rel_i} - 1$: Gain. Exponential gain is used to emphasize the difference between relevance levels.
- $\log_2(i+1)$: Discount. Relevance decreases with position i (lower-ranked results are penalized more).
- $IDCG_p$: The ideal maximum level of DCG (perfect list).
- cosine_ndcg@10 : Meaning nDCG is calculated on $p = 10$ results, using cosine similarity as the basis for calculation.

MAP (Mean Average Precision): MAP is a metric measuring the Mean Average Precision over multiple queries/sets, focusing on how many relevant items are found and at which positions they are found. This metric highly rates systems that place many correct results at the top positions. MAP is the mean value of AP (Average Precision) over all queries Q :

$$MAP = \frac{1}{|Q|} \sum_{j=1}^{|Q|} AP_j \quad (6)$$

AP (Average Precision) for a specific query j is calculated as follows:

$$AP_j = \frac{1}{R} \sum_{k=1}^n P(k) \cdot rel(k) \quad (7)$$

Where:

- $|Q|$: Total number of questions (queries) in the evaluation dataset.
- R : Total number of actually relevant documents in the database for query j .
- n : Number of documents retrieved by the system (e.g., top k).
- $P(k)$ (Precision at k): Precision at cutoff k . The ratio of correct results within the first k results.

- $rel(k)$: Indicator function of relevance at position k .
 - $rel(k) = 1$ if the document at position k is a correct result (relevant).
 - $rel(k) = 0$ if the document at position k is incorrect (not relevant).

Accuracy@ k measures the accuracy rate at the first position, reflecting the system's ability to provide the correct result immediately. This is the most important metric for end-users as they often only look at the first result. Essentially, Accuracy@1 is equivalent to Hit@1, but expressed in simpler terms:

$$\text{Accuracy@}k = \frac{\text{Number of questions with correct result within top } k}{\text{Total number of test questions}} \quad (8)$$

Where:

- Numerator is the number of questions where the system returns the correct result immediately at the first position (for Accuracy@1) or appears within the list of the first k results (for Accuracy@ k).
- Denominator is the total number of questions used to evaluate the system.

Accuracy@1 indicates ideal performance when the system operates perfectly. Hit@ K measures success probability at different levels, and MRR provides a comprehensive assessment of ranking quality by accounting for the exact position of the correct result.

Precision: That measures the "purity" of the result list returned by the system. It indicates the percentage of documents that are actually relevant to the user's query among the total documents retrieved. Precision focuses on the elimination of noise (redundant or incorrect information). Consequently, a system with high Precision implies that the user encounters minimal irrelevant content (or "junk information"), thereby reducing the effort required to process the results..

$$P@k = \frac{|\{d_i \in R_k \mid rel(d_i) = 1\}|}{|R_k|} \quad (9)$$

Written as a summation

$$P@k = \frac{1}{k} \sum_{i=1}^k rel(i) \quad (10)$$

Where:

- k : The number of documents retrieved and returned by the system.
- R_k : The set of k retrieved documents..
- $rel(i)$: The function evaluating the relevance of the document at position i ..
 - $rel(i) = 1$: If the document is relevant..
 - $rel(i) = 0$: If the document is non-relevant.

Recall: That measures the system's ability to encompass all relevant information. It addresses the question: "Out of the total number of relevant documents existing in the database, what percentage has the system successfully retrieved General Formula:

$$R@k = \frac{|\{d_i \in R_k \mid rel(d_i) = 1\}|}{|\text{Total Relevant Documents}|} \quad (11)$$

Simply written:

$$R@k = \frac{\text{Number of relevant documents in the } k}{\text{Total number of relevant documents in the Database}} \quad (12)$$

Where:

- R_k : The set of k documents returned by the system.
- $rel(d_i)$: Equals 1 if the document is relevant, 0 if non-relevant.
- Total Relevant Documents (Ground Truth): The total number of text passages within the entire database that actually contain the answer to that query.

Experimental Setup: To ensure optimal model performance, we established two distinct experimental environments to train on the two different datasets. The detailed hardware configurations and hyperparameters for each environment are presented below

Method 1: The training process for the embedding model utilizes a dataset comprising 23,628 valid *query-positive pairs*. To ensure objectivity, the data was randomly partitioned according to an 80/10/10 strategic ratio, corresponding to the Training set (18,902 samples), Validation set (2,363 samples), and Test set (2,363 samples). The experimental environment was established on the Kaggle platform utilizing an NVIDIA Tesla T4 GPU (16GB VRAM). Detailed configuration parameters for the training process are presented in Table 2.

Table 2: Hyperparameter Configuration - Method 1.

Parameter	Value	Description / Note
Batch Size	8	Configuration from code
Max Sequence Length	256 tokens	Input sequence length limit
Epochs	8	Number of training loops
Learning Rate	2×10^{-5}	Learning rate (2e-5)
Warmup Ratio	0.1	10% of steps for stabilization
Loss Function	MNR Loss	MultipleNegativesRankingLoss
Precision	AMP (FP16)	Mixed Precision via Tensor Cores
Evaluation Steps	200	Periodic evaluation

Method 2: The training process utilizes an expanded dataset comprising 65,539 *query-positive pairs*. Similar to the previous method, the data was partitioned according to an 80/10/10 ratio, corresponding to the Training set (52,431 samples), Validation set (6,554 samples), and Test set (6,554 samples). The experimental environment was maintained on the Kaggle platform using an NVIDIA Tesla T4 GPU (16GB VRAM). Detailed configuration parameters for this training process are presented in Table 3.

Table 3: Hyperparameter Configuration - Method 2.

Parameter	Value	Description / Note
Batch Size	4	Configuration from code
Max Sequence Length	512 tokens	Increased context length
Epochs	8	Number of training loops
Learning Rate	2×10^{-5}	Learning rate (2e-5)
Warmup Ratio	0.1	10% of steps for stabilization
Loss Function	MNR Loss	MultipleNegativesRankingLoss
Precision	AMP (FP16)	Mixed Precision via Tensor Cores
Evaluation Steps	200	Periodic evaluation

5.2 Fine tune results on models

Results of the first method:



Figure 8: Chart comparing actual performance before and after fine-tuning the intfloat/multilingual-e5-base model

Experimental results demonstrate it can be seen that: Accuracy@1 increased from 88.9% to 89.5% (an improvement of 0.6%), demonstrating the capability to accurately locate the answer right at the first result. MRR (Mean Reciprocal Rank) increased by 0.3% to the level of 93.8%, reflecting that the average rank of the correct result has been pushed higher. Accuracy@5 reached an excellent level of 99.2%. This is a crucial metric for RAG systems, ensuring the context information provided to the LLM almost always contains the correct answer. Although the magnitude of numerical improvement is not very large as the base model foundation was already trained on a massive dataset, the Fine-tuning process delivers two important practical values. **System Resource Optimization:** With an Accuracy@1 reaching 89.5%, the system only needs to retrieve the Top-3 chunks to achieve equivalent efficiency. This helps reduce the number of chunks requiring processing by up to 40%, thereby saving computational costs and reducing application latency. The increase in ranking metrics (such as NDCG) demonstrates that the model has learned the distinct semantic features of the Vietnamese tourism domain, helping to distinguish better between official definitions and illustrative examples in the documents.

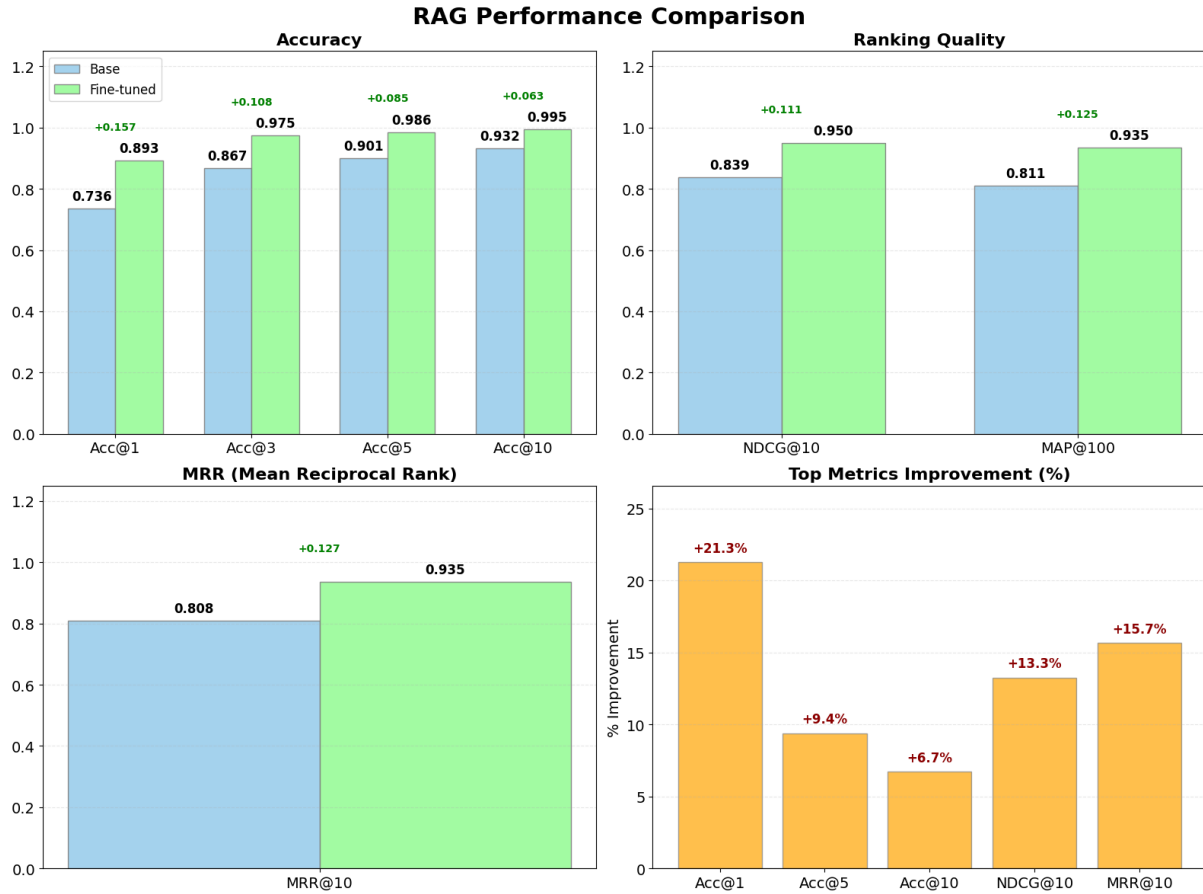


Figure 9: Chart comparing actual performance before and after fine-tuning the model bkai-foundation-models/vietnamese-bi-encoder

Experimental results demonstrate that the fine-tuned BKAI model has achieved a significant performance breakthrough compared to the base model across all evaluation metrics, validating the effectiveness of the specialized training strategy. The most notable highlight is the substantial increase in Accuracy@1, rising from 73.60% to 89.30% (an improvement of 15.70%); this implies that in practical operation, nearly 90% of user queries will instantly retrieve the most accurate reference document, significantly optimizing the Chatbot's response latency. Furthermore, with Accuracy@5 reaching a near-perfect 98.62% and ranking metrics (MRR, NDCG) both exceeding the 93% threshold, the system has demonstrated absolute reliability in covering and prioritizing correct information, ensuring the provision of high-quality context for the Large Language Model (LLM) to effectively mitigate hallucinations and enhance the overall user experience.

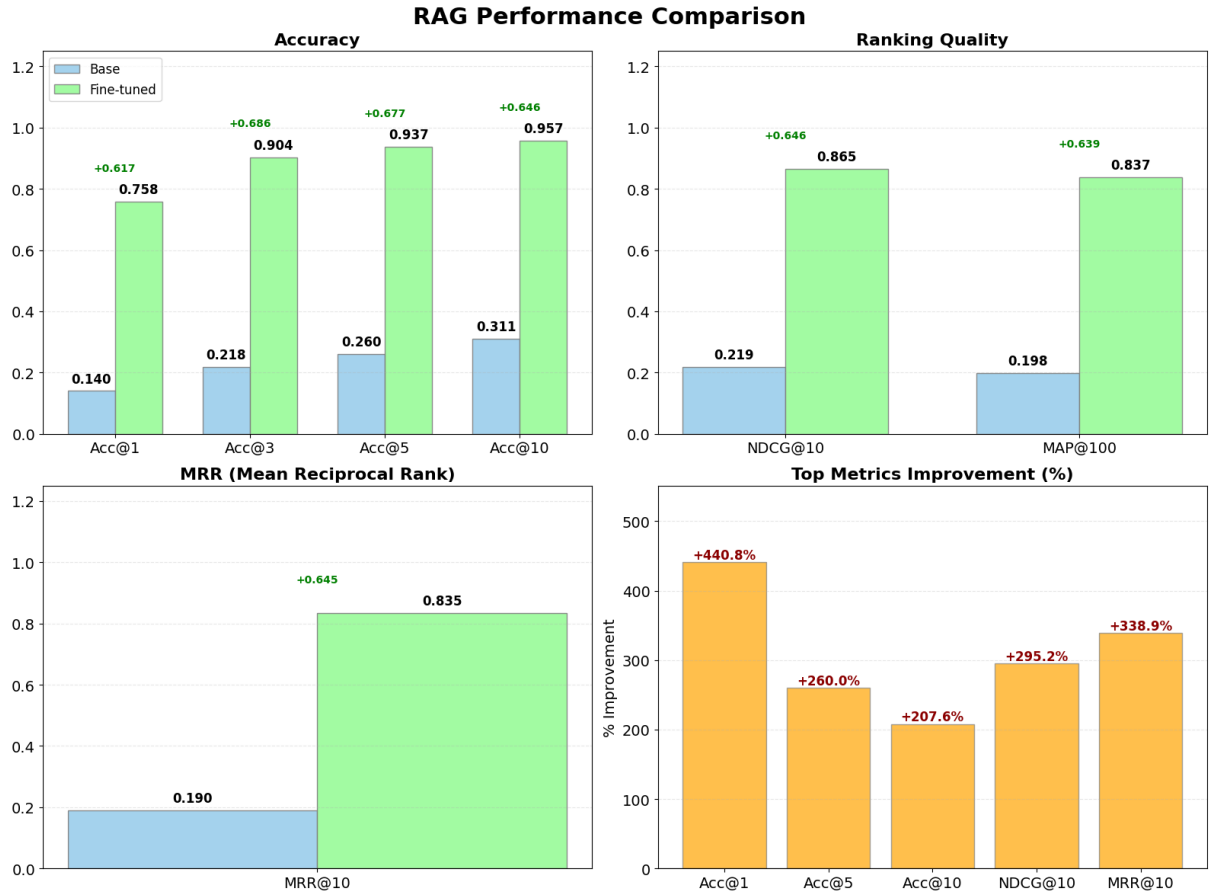


Figure 10: The chart comparing the actual performance of the all-mpnet-base-v2 model before and after fine-tuning

The experimental results on the first dataset clearly demonstrate that fine-tuning the MPNet model yields substantial performance gains across all information retrieval evaluation metrics. In particular, Accuracy@1 increases from 0.140 to 0.758, corresponding to a remarkable +440.8% improvement, showing that the model has learned to accurately match user queries with the most relevant text chunk at the top position. Similarly, Accuracy@3 rises from 0.218 to 0.904, Accuracy@5 from 0.260 to 0.937 (+260.0%), and Accuracy@10 from 0.311 to 0.957 (+207.6%), reflecting consistent enhancements in ranking depth and retrieval reliability.

Ranking-quality metrics also show strong improvements: NDCG@10 increases from 0.219 to 0.865 (+295.2%), and MAP@100 increases from 0.198 to 0.837, indicating that the fine-tuned model produces more relevant and better-ordered retrieval results. The MRR@10 metric jumps from 0.190 to 0.835 (+338.9%), confirming that correct answers appear much earlier in the ranked list.

Overall, the fine-tuned MPNet model adapts effectively to the dataset's characteristics and query patterns. The substantial improvements across all metrics validate the training strategy, model configuration, and the chosen 150-word chunking method, making the model a robust foundation for RAG pipelines and question-answering applications in real-world scenarios.

Results of the second method:

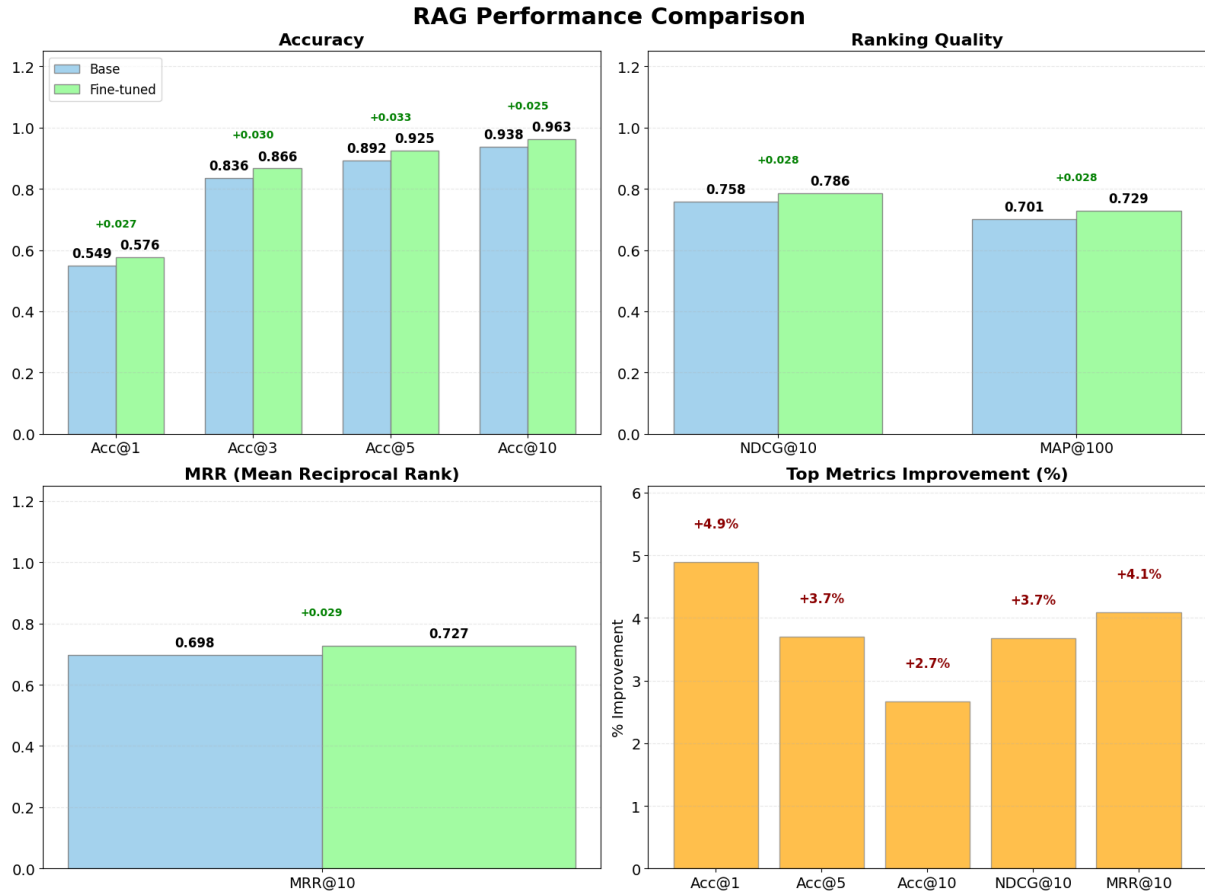


Figure 11: Chart comparing actual performance before and after fine-tuning the intfloat/multilingual-e5-base model

Experimental results demonstrate it can be seen that regarding accuracy (Accuracy & Precision), the Fine-tuned model improves the capability of correct prediction right at the first position. Specifically, the Accuracy@1 (and Precision@1) metric increased from 0.549 to 0.576, corresponding to an improvement of +4.9%. This has significant implications for minimizing noise for the RAG system. Regarding coverage capability, at the threshold of $k = 10$, the Recall@10 metric reached 0.963. This ensures that in 96.3% of cases, the system retrieves documents containing correct information within the top 10 results, maximizing the reduction of the risk of missing information. Regarding ranking quality, ranking metrics such as NDCG@10 and MRR@10 both recorded impressive growth levels (respectively +3.5% and +3.8%). MRR@10 increased from 0.701 to 0.727. NDCG@10 increased from 0.760 to 0.786. Improving these metrics proves that the Fine-tuned model has the ability to "push" the most relevant documents to the top positions, helping to optimize the input context for the Large Language Model. The average improvement level is approximately 4% on Top-1 metrics (Acc@1, P@1, R@1).

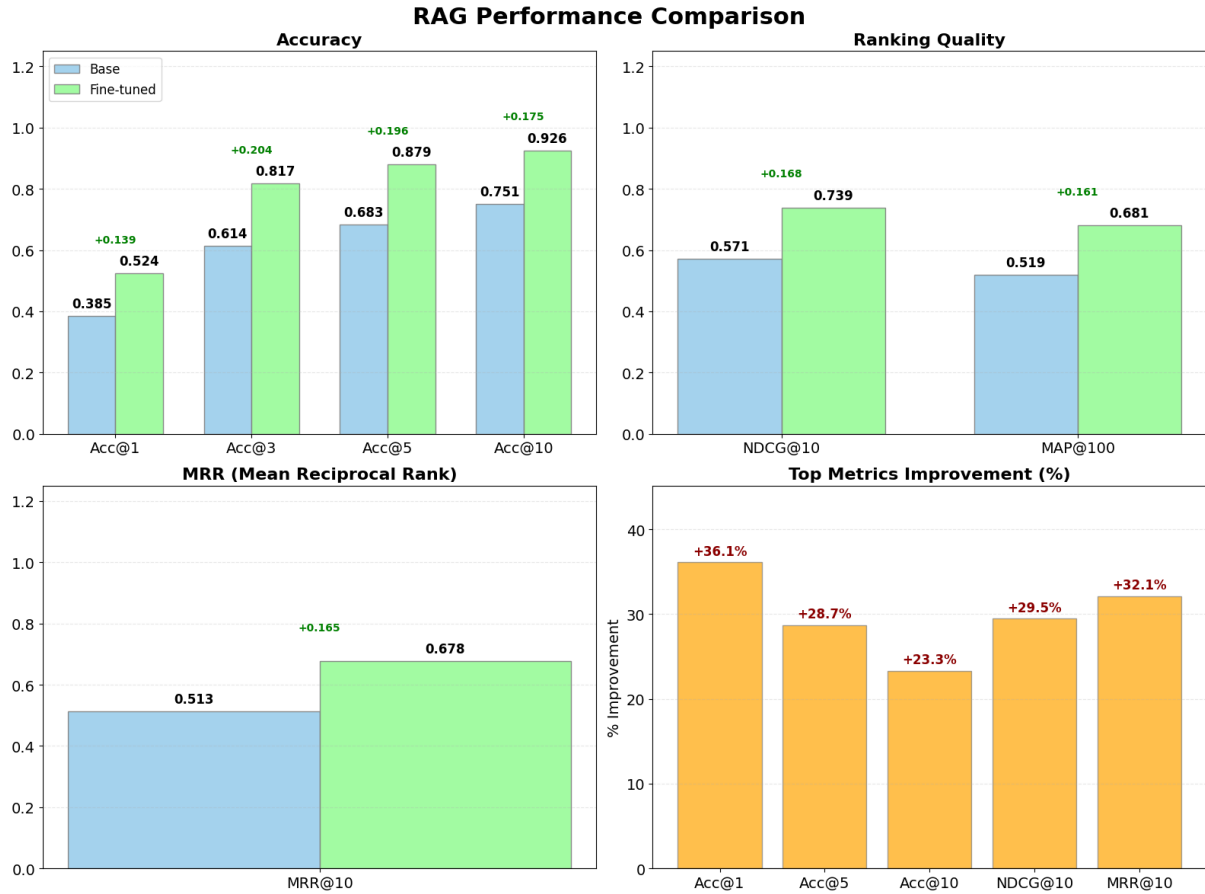


Figure 12: Chart comparing actual performance before and after fine-tuning the model bkai-foundation-models/vietnamese-bi-encoder

Experimental results on the dataset demonstrate the effectiveness of the fine-tuning process in handling complex queries. Although the increased data complexity caused a substantial decline in the base model's performance (with Accuracy@1 reaching only 38.51.%), the fine-tuned model achieved a 13.89% improvement in first-position accuracy and a 19.63% increase in Top-5 information coverage (reaching 87.92%). Concurrently, ranking quality metrics such as MRR@10 and NDCG@10 exhibited remarkable growth (+16.47% and +16.83%, respectively). This confirms that the model not only retrieves information more effectively but also ranks results more intelligently, ensuring the provision of high-quality context for the system even in scenarios involving highly noisy input data.

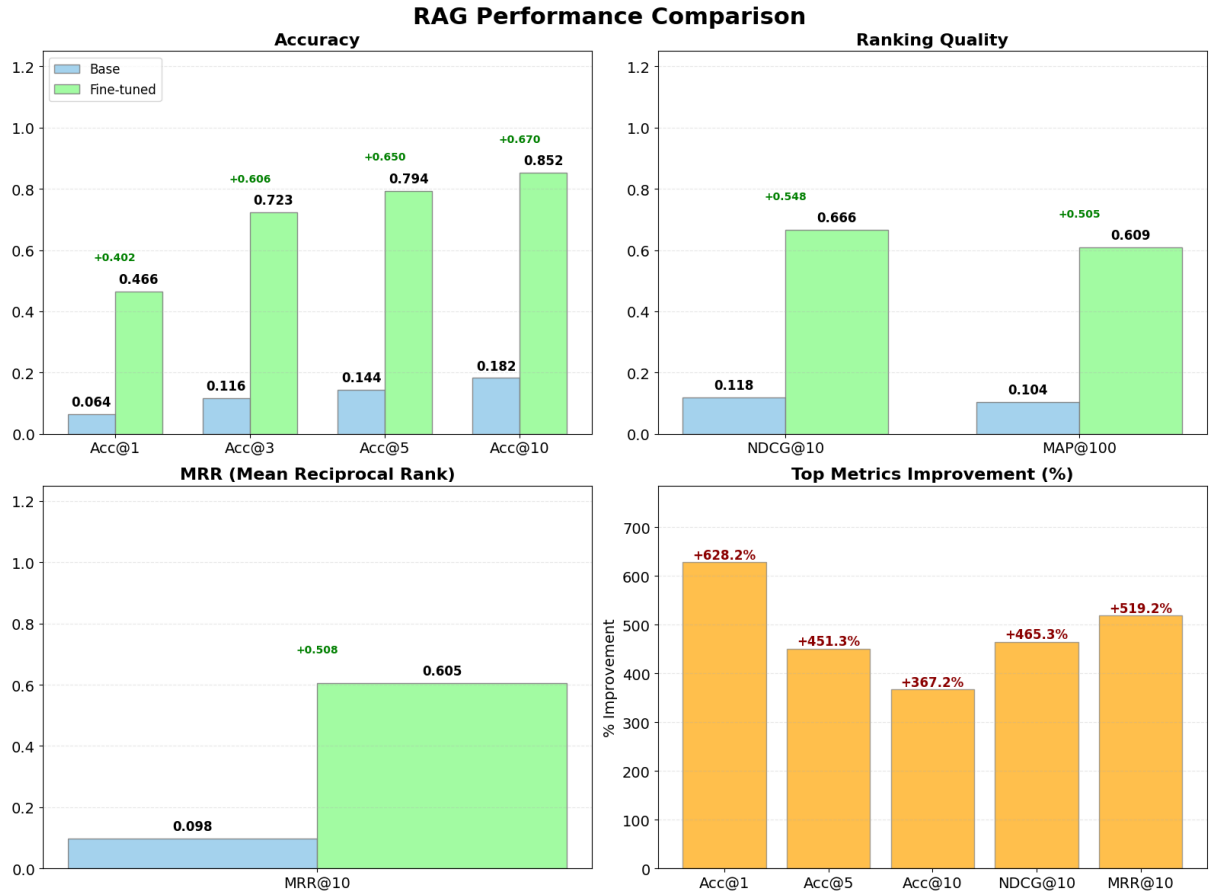


Figure 13: The chart comparing the actual performance of the all-mpnet-base-v2 model before and after fine-tuning

The evaluation results on the second dataset demonstrate that fine-tuning MPNet yields substantial and systematic performance gains across all retrieval metrics. Accuracy shows marked improvements, with Acc@1 increasing from 0.064 to 0.466 (+628.2%), Acc@5 from 0.144 to 0.794 (+451.3%), and Acc@10 from 0.182 to 0.852 (367.2%). Ranking-based metrics exhibit similar trends: NDCG@10 rises from 0.118 to 0.666 (+465.3%), while MAP@100 improves from 0.104 to 0.609, indicating significantly enhanced relevance estimation and ordering. Furthermore, MRR@10 increases from 0.098 to 0.605 (+519.2%), confirming that the fine-tuned model retrieves correct answers at substantially higher ranks.

Overall, the improvements ranging from +367% to over +620% underscore the effectiveness of the fine-tuning procedure, showing that MPNet adapts well to the semantic characteristics of the dataset and provides a more accurate and reliable foundation for RAG-based retrieval and question-answering tasks.

Selection of the Embedding Model for the RAG System: Based on the empirical evaluation results, we have decided to select the fine-tuned version of the BKAI-Vietnamese-Bi-Encoder as the core embedding model for the RAG system. This decision is driven by the following key factors:

- **High Domain Adaptation Capability:** Although E5-Base achieved a slightly higher absolute score (0.996), its post-tuning improvement was marginal, suggesting that the model may have reached saturation or lacks sensitivity to the specific domain data. In contrast, BKAI demonstrated superior learnability, witnessing a substantial growth of +15.70 (from 0.736 to 0.893) in Acc@1. This significant surge confirms that the embedding space of BKAI has been successfully and effectively aligned with the semantic context of the query-document pairs in our dataset.
- **Competitive Absolute Performance and Stability:** The final Acc@1 score of BKAI (0.893) is comparable to that of the leading model (E5-Base). Furthermore, critical ranking quality metrics such as NDCG@10 (0.950) and MRR@10 (0.935) exhibit excellence. These indicators ensure that the most relevant documents are consistently ranked at the top positions, directly enhancing the retrieval accuracy for the generation phase.
- **Native Optimization for Vietnamese Language:** BKAI-Vietnamese-Bi-Encoder is specifically pre-trained on a comprehensive Vietnamese corpus. Utilizing a native model guarantees superior performance in handling

linguistic nuances, domain-specific terminology, and language-specific tokenization challenges. This offers a distinct advantage over multilingual models (such as MPNET or E5), which may encounter limitations when processing complex Vietnamese structures.

Conclusion: The fine-tuned version of the BKAI-Vietnamese-Bi-Encoder is officially selected for the RAG pipeline, representing the optimal balance between absolute retrieval performance and domain adaptability.

6 Construction of Vector Database and Storage System

6.1 Select vector database

In this project, ChromaDB [32] was selected as the specialized vector storage solution, serving as the backbone of the RAG service. Its primary function is to manage and store high-dimensional feature vectors (embeddings) extracted from document chunks. With optimization for *similarity search* tasks, ChromaDB ensures high-performance querying, playing a pivotal role in the *retrieval phase* of the RAG pipeline.

6.2 Data Structure and Organization

The input dataset, comprising 25,153 chunks, is normalized into JSONL format and mapped into ChromaDB. To ensure semantic integrity and traceability, each record is structured into four parallel components:

- Primary ID: A unique identifier for each text segment, facilitating CRUD (Create, Read, Update, Delete) operations.
- Document: Contains the original text content (*passage*). This raw text serves as the context for the Large Language Model following the retrieval step.
- Metadata: Stores auxiliary information in a dictionary format, including the title (`title`), length (`len`), and the opening sentence (`line_first_sentence`). This component supports filtering search results based on specific criteria.
- Embedding Vector: The vector representation of the text segment in a high-dimensional space, extracted using the fine-tuned *Bi-Encoder* model.

```
{
  "id": "1",
  "document": "An Giang mùa nào đẹp. Ở An Giang...",
  "metadata": {
    "doc_id": "1",
    "title": "output_paragraph_1",
    "len": 120
  },
  "embedding": [0.012, -0.931, 0.442, ...]
}
```

Figure 14: Logical structure of a record in ChromaDB

6.3 Interaction History Storage (MongoDB):

Complementing the Vector Database, the system integrates an unstructured storage layer (NoSQL) using MongoDB to manage interaction history. This component is responsible for recording user sessions, enabling behavioral analysis, conversational context reconstruction, and system debugging. The selection of MongoDB is driven by its flexibility in handling semi-structured data (JSON). It supports dynamic schema expansion, efficient indexing by time or `user_id`, and seamless integration with the Python ecosystem via the PyMongo library.

7 System Implementation

The operational workflow of the RAG system is structured as a sequential pipeline comprising three primary stages: Query Processing, Data Retrieval, and Response Generation.

7.1 Query Processing

Prior to data retrieval, the system implements two preprocessing steps aimed at optimizing input quality and enhancing user experience:

- **Intent Classification:** The system utilizes the *gpt-4o-mini* language model to analyze the semantic intent of the user's question. For inquiries categorized as social interaction ("small talk"), such as greetings or expressions of gratitude, the system triggers immediate responses using predefined friendly templates. This mechanism bypasses the computationally expensive vector search process, thereby conserving computational resources and reducing response latency. The specific prompt engineering for this classification task is illustrated in Figure ??.

```
prompt = f"""
### Vai trò:
Bạn là một bộ lọc nội dung thông minh cho hệ thống Chatbot Du lịch Việt Nam. Nhiệm vụ cốt lõi của bạn là phân tích ý định của người dùng (User Intent) để phân biệt giữa "Giao tiếp xã giao" (Small Talk) và "Truy vấn du lịch".

### Định nghĩa:
1. Truy vấn du lịch: Các câu hỏi liên quan đến địa điểm, ăn uống, lịch trình, thời tiết, văn hóa hoặc chi phí tại Việt Nam.
2. Small Talk: Các câu chào hỏi, trêu đùa, câu hỏi về bản thân chatbot, hoặc các chủ đề không liên quan (ví dụ: code, toán học...).

### Quy tắc xử lý:
1. Trường hợp là Truy vấn du lịch:
- Chỉ trả về duy nhất từ khóa: "no".
2. Trường hợp là Small Talk:
- Hãy đóng vai hướng dẫn viên ảo. Phản hồi thân thiện, giới thiệu ngắn gọn về chatbot và mời người dùng đặt câu hỏi du lịch.
- Ngôn ngữ phản hồi: tiếng việt

### Dữ liệu mẫu:
User query: "Hello, nice to meet you."
Response: "Chào mừng bạn! Mình là trợ lý AI chuyên về du lịch Việt Nam. Bạn cần tìm thông tin gì không?"

User query: "Vé vào cửa Bà Nà Hills giá bao nhiêu?"
Response: "no"

User query: "Viết cho tôi đoạn code Python."
Response: "Rất tiếc, mình chỉ chuyên về du lịch Việt Nam thôi. Nhưng mình có thể gợi ý quán cà phê yên tĩnh để bạn ngồi code!"

User query: "Tư vấn lịch trình 3 ngày đi."
Response: "no"

### Input thực tế:
Câu hỏi từ người dùng: {user_input}
"""
```

Figure 15: Cấu trúc Prompt phân loại ý định người dùng (Small Talk vs. Tourism Intent)

- **Query Rewriting:** To address ambiguity in multi-turn conversations (e.g., a user asking “Where is it?” after previously mentioning “Ha Long Bay”), the system leverages the history of the last interaction turns to reconstruct the query. The result is a semantically complete “Standalone Question”, ensuring the Retriever always receives the most informative input independent of implicit context. The specialized prompt designed for this task is illustrated in Figure 16.

```
prompt = """
Vai trò: Bạn là chuyên gia "Query Rewriting". Nhiệm vụ là viết lại [Câu hỏi hiện tại] dựa
trên [Lịch sử hội thoại] để tạo thành câu hỏi độc lập (Standalone Question).

Quy tắc BẮT BUỘC:
1. Thay thế đại từ (nó, đó, ở đây...) bằng danh từ cụ thể trong lịch sử.
2. BỔ sung chủ ngữ/vị ngữ nếu câu hỏi bị cụt lủn dựa trên ngữ cảnh trước đó.
3. Nếu câu hỏi đã rõ ràng hoặc ĐỐI CHỦ ĐỀ MỚI -> Giữ nguyên câu hỏi gốc.
4. KHÔNG trả lời câu hỏi, chỉ viết lại.
5. Giữ nguyên ngôn ngữ Tiếng Việt.

Ví dụ:
Lịch sử: user: Vịnh Hạ Long ở đâu? | assistant: Ở Quảng Ninh.
Câu hiện tại: Vé ở đó bao nhiêu?
-> Output: Vé tham quan Vịnh Hạ Long bao nhiêu tiền?
Lịch sử: user: Món ngon Hà Nội? | assistant: Phở, bún chả.
Câu hiện tại: Sài Gòn có gì vui?
-> Output: Sài Gòn có gì vui?

Input thực tế:
[Lịch sử hội thoại]: {text_history}
[Câu hỏi hiện tại]: {current_question}

Output:
"""
```

Figure 16: System Prompt for Contextual Query Rewriting

7.2 Retrieval and Context Reconstruction Module

The information retrieval process is executed through two consecutive phases to ensure the Precision and Coherence of the input data provided to the generative model:

- **Vector Retrieval (Dense Retrieval):** Once normalized, the query is encoded by the fine-tuned Bi-Encoder model. Subsequently, the system utilizes the *Cosine Similarity* metric within the vector space to retrieve the top- K most relevant text chunks (with $K = 10$) from the ChromaDB database.
- **Context Smoothing:** The raw text chunks obtained from retrieval undergo a post-processing step using the Context Smoothing algorithm to enhance corpus quality. This procedure involves four consecutive optimization steps: first, merging text chunks with adjacent indices in the original document to rectify fragmentation caused by chunking; next, an adaptive filtering mechanism removes segments with low similarity based on score gaps to minimize misinformation. Subsequently, the system retrieves and appends preceding and succeeding sentences from the original corpus to ensure the context is semantically complete; finally, a deduplication step retains only the most representative text segment for each document to maximize input diversity.

7.3 Response Generation

Upon synthesizing the smoothed and optimized context set, the system proceeds to the final step of response generation:

- **Prompt Engineering Strategy:** We designed a specialized system prompt to define the Chatbot’s persona as an “Intelligent Travel Assistant”. This prompt enforces strict constraints to ensure *contextual adherence* and mitigate *hallucinations*. Specifically, the model is instructed to derive answers solely from the retrieved data, maintain a hospitable tone, and explicitly acknowledge data limitations when the provided context is insufficient. The specific prompt structure is illustrated in Figure 17.

```
PROMPT_TEMPLATE = (
Vai trò: Bạn là trợ lý du lịch ảo thông minh, am hiểu sâu sắc về du lịch Việt Nam. Nhiệm vụ của bạn là hỗ trợ du khách dựa trên thông tin được cung cấp. Nguyên tắc trả lời:

1. Phân tích sâu: Hãy đọc kỹ các ngữ cảnh được cung cấp (context). Đừng chỉ tìm từ khóa chính xác, hãy tìm các ý liên quan, đồng nghĩa hoặc suy luận logic từ ngữ cảnh để trả lời câu hỏi.
2. Trả lời linh hoạt:
- Nếu ngữ cảnh chứa đầy đủ thông tin: Trả lời chi tiết, chính xác.
- Nếu ngữ cảnh chỉ chứa một phần thông tin: Hãy trả lời dựa trên những gì có trong ngữ cảnh và nói rõ rằng hệ thống chưa có thông tin chi tiết cho phần còn lại. Đừng từ chối toàn bộ câu hỏi.
3. Phong cách: Thân thiện, nhiệt tình như một hướng dẫn viên du lịch bản địa.
4. Xử lý thiếu tin: Chỉ khi ngữ cảnh HOÀN TOÀN KHÔNG LIÊN QUAN đến câu hỏi, bạn mới được phép trả lời: "Hiện tại cơ sở dữ liệu của tôi chưa cập nhật thông tin chính xác về vấn đề này...".
5. Ngôn ngữ: Trả lời bằng ngôn ngữ: {language}

Dữ liệu được truy xuất: {input}
Câu hỏi của người dùng: {question}
Câu trả lời của bạn: )
```

Figure 17: Configuration of the System Prompt for the Travel Assistant Persona

- **Response Generation Mechanism:** The retrieved context segments, combined with the normalized user query, are fed into the Large Language Model (*gpt-4o-mini*). The model synthesizes this information to generate natural, accurate, and context-aware responses in Vietnamese, thereby completing the RAG pipeline.

7.4 System Demonstration

The actual system interface is illustrated in Figure 18. The design is optimized for user experience, featuring a layout divided into three main functional areas:

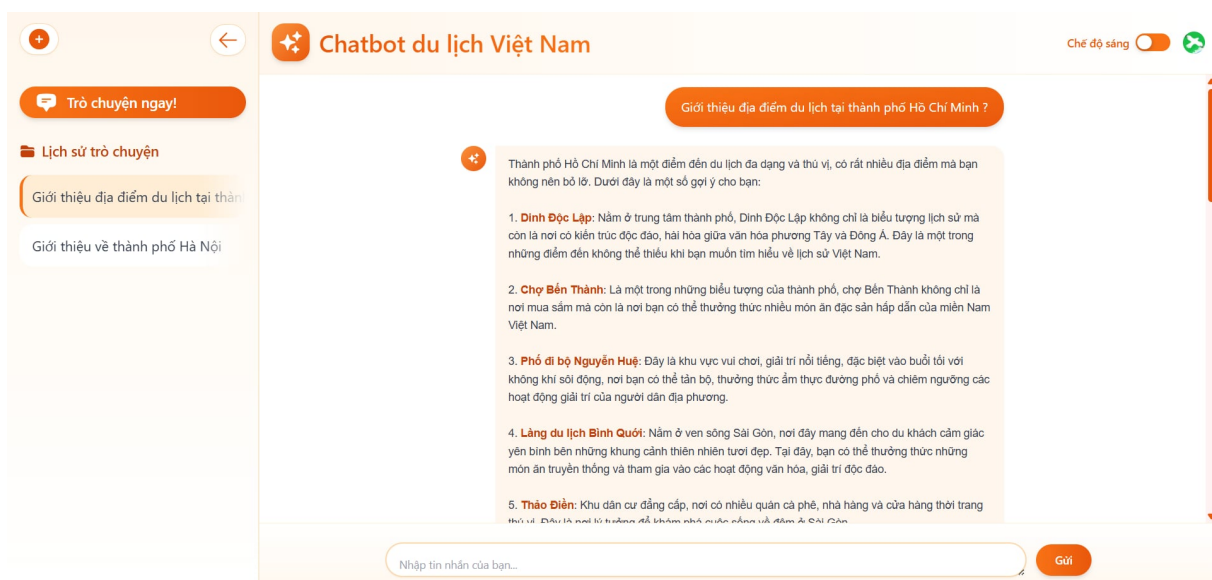


Figure 18: Chatbot User Interface.

- **Left Sidebar (Navigation):** This area facilitates session management, featuring a "+" Button (Top left corner) that allows users to initiate a completely new chat session (New Chat), clearing the context of previous conversations. Additionally, the "Chat History" section displays a list of saved sessions (e.g., "Introduction to Ho Chi

Minh City", "Introduction to Ho Noi City"), enabling users to review or continue previous conversations, which demonstrates the system's stable state management capabilities.

- **Top Toolbar (Header):** This section clearly identifies the application with the title "Vietnam Tourism Chatbot". It also includes a "Dark Mode" Toggle in the top right corner, allowing for flexible switching between light and dark themes to help protect user eyesight during prolonged use.
- **Main Chat Interface:** This is the core interaction zone where the Conversation Flow visually displays user queries (right-aligned) and AI responses (left-aligned). In the figure, the system is shown responding to a query about "Entertainment spots in Hanoi" with a detailed list based on the Retrieval Augmented Generation mechanism. At the bottom, the Input Field (placeholder "Enter your message...") allows users to input questions using natural language, while the "Send" Button (Blue paper plane icon) activates the processing pipeline.

8 Conclusion

In this work, we introduced a smart Chatbot system for Vietnam tourism, integrating an advanced Retrieval-Augmented Generation (RAG) architecture to address the challenges of data fragmentation and information hallucination. Our proposed utilization of the fine-tuned BKAI-Vietnamese-Bi-Encoder embedding model, seamlessly combined with sliding window chunking and Cross-Encoder noise filtering strategies, effectively captures subtle semantic nuances and specific local entities while reducing reliance on surface-level keywords. Comprehensive evaluations demonstrated that the system achieves superior detection and retrieval performance, with Accuracy@1 surging to 89.03% and Accuracy@5 reaching a near-perfect 98.62%, significantly surpassing the base model and other baseline methods. Moreover, ranking evaluations further showcased the system’s strong generalization capability, achieving MRR@10 and NDCG@10 scores both exceeding 93%—a result that confirms the model’s effectiveness in handling complex queries containing rich cultural information without excessive manual intervention. This performance improvement underscores the validity of the specialized training strategy in mitigating semantic discrepancies compared to previous multilingual approaches, aligning closely with our objective to enhance reliability and user experience. Ablation studies also highlighted the complementary benefits of integrating ChromaDB and MongoDB, emphasizing the effectiveness of the intelligent query processing workflow, which led to optimal response latency and the highest quality input context for the generative model.

However, despite its notable performance, the system’s adaptability under challenging real-world conditions—such as real-time fluctuating data (e.g., ticket prices, weather) and multimodal interaction needs—remains limited and warrants further investigation. In future work, we plan to address these vulnerabilities by enhancing the model’s resilience through automated data update mechanisms (auto-crawling) and exploring deeper personalization strategies. These strategies would dynamically adjust to user behaviors and preferences during inference, aiming to achieve more flexible, comprehensive, and resilient tourism support capable of meeting the increasingly diverse needs of travelers.

References

- [1] Vietnam National Authority of Tourism: Vietnam tourism: 2023 year in review. <https://vietnamtourism.gov.vn/en/post/19448>. Accessed: 2025-10-20 (2024)
- [2] General Statistics Office of Vietnam: Socio-economic situation in 10 months of 2024. <https://www.gso.gov.vn/en/data-and-statistics/2024/10/socio-economic-situation-in-10-months-of-2024/>. Accessed: 2025-11-06 (2024)
- [3] Beautiful Destinations: Layla: The AI Trip Planner created by Beautiful Destinations and Roam Around. <https://justasklayla.com/>. Accessed: 2025-10-20 (2024)
- [4] Jannach, D., et al.: Challenges and opportunities in travel recommendation systems. arXiv preprint arXiv:2209.04265 (2022)
- [5] Calvaresi, D., et al.: Chatbots for tourism and hospitality: A systematic literature review. *Information and Communication Technologies in Tourism 2021*, 165–179 (2021)
- [6] Pillai, R., Sivathanu, B.: Adoption of ai-based chatbots for travel planning and booking: The moderating role of trust and perceived risk. *Journal of Travel Research* **59**(5) (2020)
- [7] Sperlí, G.: Unsupervised chatbot construction for the cultural heritage domain. In: 2018 IEEE International Conference on Big Data (Big Data), pp. 2677–2680 (2018). IEEE
- [8] Buhalis, D., et al.: Generative artificial intelligence in tourism: A review and future research directions. *Tourism Review* (2023). EarlyCite
- [9] Li, Y., et al.: Chatdoctor: A medical chat model fine-tuned on llama model using medical domain knowledge. arXiv preprint arXiv:2303.14070 (2023)
- [10] Wu, S., et al.: Bloomberggpt: A large language model for finance. arXiv preprint arXiv:2303.17564 (2023)
- [11] Kasneci, E., et al.: Chatgpt for good on opportunities and challenges of large language models for education. *Learning and Instruction* **85**, 101772 (2023)
- [12] Rezayi, S., et al.: Agribert: Knowledge-based agricultural question answering system. In: *Proceedings of the CIKM 2022 Workshops*, vol. 3318, pp. 1–9. CEUR-WS.org, Aachen (2022)
- [13] Nguyen, D.Q., et al.: A survey of vietnamese natural language processing. arXiv preprint arXiv:2301.01358 (2023)
- [14] Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474 (2020)
- [15] Zhang, Y., et al.: Siren’s song in the ai ocean: A survey on hallucination in large language models. arXiv preprint arXiv:2309.01219 (2023)
- [16] VNExpress: Du lịch. <https://vnexpress.net/du-lich>. Accessed: 2025-01-12
- [17] Vietnam National Administration of Tourism: Vietnamtourism. <https://vietnamtourism.gov.vn>. Accessed: 2025-01-12
- [18] iVIVU: Cẩm nang du lịch. <https://www.ivivu.com/blog>. Accessed: 2025-01-12
- [19] Dulichviet: Tin tức du lịch. <https://dulichviet.com.vn/tin-tuc-du-lich>. Accessed: 2025-01-12
- [20] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.-t.: Dense passage retrieval for open-domain question answering. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781 (2020)
- [21] Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2019)
- [22] Song, S., Yan, X., Tan, F., Zhang, W., Ren, X., Liu, X.: MpNet: Masked and permuted pre-training for language understanding. arXiv preprint arXiv:2004.09297 (2020)
- [23] Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992 (2019)
- [24] Izacard, G., Grave, E.: Leveraging passage retrieval with generative models for open domain question answering. In: *ACL 2021* (2021). <https://arxiv.org/abs/2007.01282>
- [25] Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., Wei, F.: Multilingual e5 text embeddings: A technical report. arXiv preprint arXiv:2402.05672 (2024)
- [26] Nguyen, Q.D., et al.: Towards comprehensive vietnamese retrieval-augmented generation and large language models. arXiv preprint arXiv:2403.01616 (2024)
- [27] Nguyen, D.Q., Tuan, N.A.: Phobert: Pre-trained language models for vietnamese. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1037–1042 (2020)
- [28] Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. arXiv preprint arXiv:1908.10084 (2019)
- [29] Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., Le, Q.V.: Xlnet: Generalized autoregressive

- pretraining for language understanding. arXiv preprint arXiv:1906.08237 (2019)
- [30] Shaw, P., Uszkoreit, J., Vaswani, A.: Self-attention with relative position representations. arXiv preprint arXiv:1803.02155 (2018)
 - [31] Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press, Cambridge, UK (2008). <https://nlp.stanford.edu/IR-book/>
 - [32] Pan, J., Wang, Y., Li, C.: A comprehensive survey on vector database: Storage, indexing, and query processing. arXiv preprint arXiv:2310.11703 (2023)